

Problem

The QR currently encodes `...?u=<uuid>&s=<signature>` in cleartext. Anyone with a generic scanner sees the UUID and signature. These values are sensitive (they drive the Stage-1 signature gate) and should not be human-readable or enumerable from a raw scan.

Goal

Replace the readable `u/s` params with a single **opaque token**, while preserving: (a) normal-scanner tap → store/app redirect, and (b) the existing signature-gate verification logic.

Out of scope: This does **not** stop photocopy/cloning of a genuine label. It hides and tamper-protects the payload only.

Functional Requirements

#	Requirement
FR1	QR encodes exactly one URL: <code>https://verify.digitathya.com/v/<token></code> . No <code>u/s</code> visible.
FR2	<code><token></code> is opaque — not human-decodable, not enumerable (no sequential IDs).
FR3	A generic scanner tapping the URL resolves to a real redirect (no error page).
FR4	Server detects platform via User-Agent: iOS → App Store, Android → Play Store, other → web fallback.
FR5	If the DigiTathya app is installed, the link opens the app (deep link), not the browser.
FR6	The app's verification flow recovers <code>uuid</code> + <code>signature</code> from <code><token></code> and runs the unchanged Stage-1 signature gate, then Stage-2 OCR.
FR7	A token can be revoked server-side so a known-bad label fails fast (ties to Revoked lifecycle state).
FR8	Token issuance happens at QR generation; one token ↔ one product UUID.
FR9	Existing field stock (cleartext <code>?u=&s=</code> labels) continues to verify — do not break grandfathered labels.

4. Two Options (Please treat this as suggestions)

	Option A — Random ID + lookup (recommended)	Option B — Encrypted blob
Token is	A random opaque ID	Ciphertext of <code>uuid+signature</code> (AES-GCM / signed JWT)
Server resolves by	DB lookup <code>token → uuid</code>	Decrypt in-process
Revocation (FR7)	Trivial — flip a DB row	Needs a deny-list
Key management	None in payload	Key storage + rotation required
Extra DB read/scan	Yes (one, indexed)	No
Best when	You want simple revocation + no new crypto	You want zero extra lookups

Recommendation: Option A. You already store per-UUID data in `ProductUuid.security_data`; a `token → uuid` map is a small additive table, gives free revocation, and avoids adding a *second* undocumented crypto scheme (the signature gate's algorithm/rotation is already an open eng question — don't compound it).

5. Stepwise Approach (Option A) - sequenced to keep dev simple

- Schema:** add a `qr_tokens` table (or column): `token` (indexed, unique), `product_uuid`, `status` (active/revoked), `created_at`. One migration.
- Token generation:** at QR mint, generate a random opaque token (e.g. 128-bit URL-safe), store the mapping. No change to how `signature_data` is produced.
- Redirect endpoint:** `GET /v/{token} →`
 - look up token → if missing/revoked, serve a neutral "not recognized" page;
 - else read User-Agent → 302 to App Store / Play Store; serve app-link metadata so an installed app intercepts.
- App links plumbing:** host `apple-app-site-association` (iOS) + `assetlinks.json` (Android) on `verify.digitathya.com`. This is what makes FR5 reliable.
- Verification path:** app calls verify endpoint with the token (F4: alongside the image). Server resolves token → `uuid` → loads stored `signature_data` → runs **existing** `QrsignatureService` gate → Stage-2 OCR. **No change to gate logic — only where `uuid/signature` are sourced.**
- Backwards-compat:** verify endpoint accepts *both* a token and a legacy `?u=&s=` payload (FR9). Branch on which is present.

7. **Revoke hook:** wire token `status=revoked` into the Revoked lifecycle state so the same action covers both.

Dev-life win: steps 1–2 and 3–4 are independent workstreams (backend mint vs redirect/mobile) and can run in parallel. The gate logic (step 5) is the *smallest* change — it's a source swap, not a rewrite.